

— PYTHON BASICS

Python *Control Flow*



Branching and Relational Logic

Md. Owaliur Rahman

Lecturer, Department of CSE
Presidency University



Today's Learning Goals

By the end of this class

01 Explain what **branching** means in programming.

02 Use relational operators: `==`, `!=`, `>`, `<`, `>=`, `<=`.

03 Combine conditions using **and**, **or**, and **not**.

04 Write **if**, **if-else**, and **if-elif-else** programs.



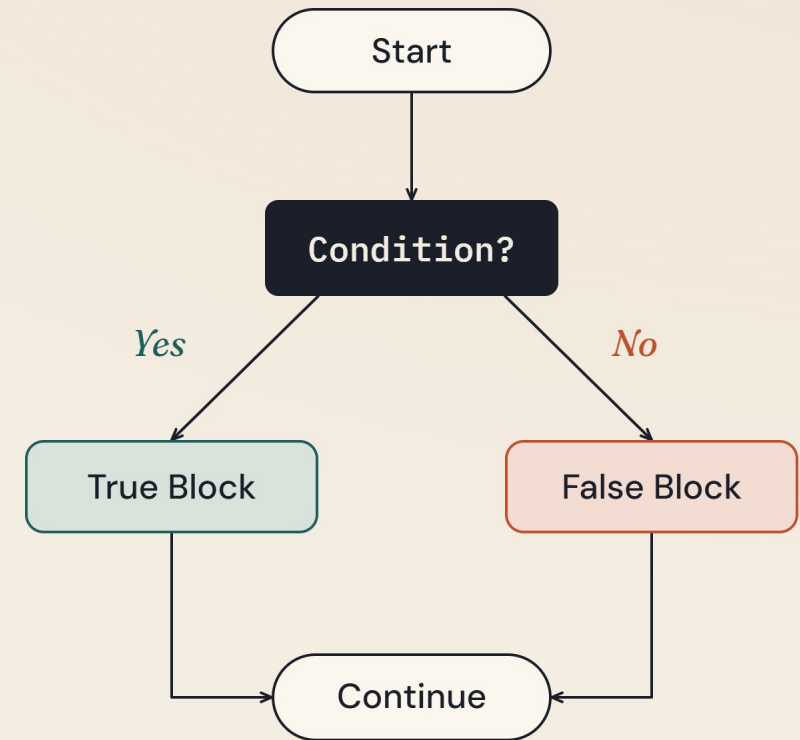
What is Branching?

Decision Making in Code

Branching allows a program to **choose different paths** depending on a condition.

Instead of always running line by line, the program asks a question:

| Is this condition *True*?





Real-Life Branching Examples

Daily Life

- If it is raining, take an umbrella.
- If the traffic light is red, stop.
- If the phone battery is below 20%, charge it.

Programming

- If **marks** ≥ 40 , print "**Pass**".
- If password is correct, allow login.
- If denominator is zero, do not divide.



Relational Operators

Before branching, we usually compare values. The result is always **True** or **False**.

SYMBOL	MEANING	EXAMPLE	RESULT
<code>==</code>	Equal to	<code>5 == 5</code>	<i>True</i>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<i>True</i>
<code>></code>	Greater than	<code>10 > 20</code>	<i>False</i>
<code><</code>	Less than	<code>10 < 20</code>	<i>True</i>
<code>>=</code>	Greater/equal	<code>5 >= 5</code>	<i>True</i>
<code><=</code>	Less/equal	<code>5 <= 4</code>	<i>False</i>



Relational Operators: Quick Examples

```
1 print(10 > 5)      # True
2 print(10 < 5)      # False
3 print(7 == 7)      # True
4 print(7 != 7)      # False
5 print(80 >= 80)     # True
6 print(39 <= 40)     # True
```

- **VERY COMMON BEGINNER MISTAKE**

= means **assignment**.

== means **comparison**.



Logical Operators

Logical operators combine multiple conditions.

A	B	A AND B	A OR B
<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>
<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>
<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>
<i>False</i>	<i>False</i>	<i>False</i>	<i>False</i>

- REMEMBER

and: all conditions must be True.

or: at least one condition must be True.

not: reverses True/False.



Logical Operators: Examples

```
1 age = 20
2 has_id = True
3
4 print(age >= 18 and has_id == True)    # True
5 print(age < 18 or has_id == True)      # True
6 print(not has_id)                      # False
```

- NATURAL LANGUAGE TRANSLATION

age >= 18 and has_id == True

means: The person is adult **and** has an ID card.



1. The **if** Statement

Basic Structure

The **if** statement runs a block of code **only if** the condition is True.

```
1 if condition:
2     # indented block
3     statement_1
4     statement_2
```

- **INDENTATION MATTERS**

Python uses indentation to understand which lines belong inside the **if** block.



if Example 1: Positive Number

```
1 num = int(input("Enter a number: "))
2
3 if num > 0:
4     print("The number is positive")
5
6 print("Program finished")
```

- TRY IT

Input **10**: prints positive message.

Input **-5**: skips the **if** block, but still prints **"Program finished"**.



if Example 2: Voltage Warning

```
1 voltage = 250
2
3 if voltage > 240:
4     print("DANGER!")
5     print("Overvoltage warning!")
```

- WHAT HAPPENS?

Since **250 > 240** is True, both indented lines run.



if Example 3: String Comparison

```
1 password = input("Enter PIN: ")
2
3 if password == "1234":
4     print("Access granted")
5     print("System unlocked")
```

- **IMPORTANT**

For text/string comparison, use quotes: **"1234"**. Without quotes, Python treats it as a number.



if Example 4: Checking Even Number

```
1 num = int(input("Enter a number: "))  
2  
3 if num % 2 == 0:  
4     print("Even number")
```

• KEY IDEA

num % 2 gives the remainder after dividing by 2.

If the remainder is 0, the number is even.



2. The **if-else** Statement

Handling Both Outcomes

Use **if-else** when there are exactly two possible directions.

```
1 if condition:
2     # runs if condition is True
3     true_block
4 else:
5     # runs if condition is False
6     false_block
```

- **SYNTAX RULE**

else does not take a condition.
It also ends with a colon.



if-else Example 1: Voting Eligibility

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("You are eligible to vote")
5 else:
6     print("You are too young to vote")
```

- **BINARY DECISION**

The person is either eligible or not eligible.
Exactly one block runs.



if-else Example 2: Pass or Fail

```
1 marks = float(input("Enter marks: "))
2
3 if marks >= 40:
4     print("Pass")
5 else:
6     print("Fail")
```

• CLASSROOM EXAMPLE

This is one of the simplest and most useful beginner examples of branching.



if-else Example 3: Safe Division

```
1 numerator = float(input("Numerator: "))
2 denominator = float(input("Denominator: "))
3
4 if denominator != 0:
5     result = numerator / denominator
6     print("Result:", result)
7 else:
8     print("Cannot divide by zero")
```

- WHY THIS MATTERS

We check the dangerous condition before doing the division.



if-else Example 4: Login Check

```
1 username = input("Username: ")
2 password = input("Password: ")
3
4 if username == "admin" and password == "1234":
5     print("Login successful")
6 else:
7     print("Invalid username or password")
```

- COMBINED CONDITION

Both username and password must match. That is why we use **and**.



3. The **if-elif-else** Statement

Multiple Conditions

Use **elif** when there are more than two possible outcomes. Python checks conditions from top to bottom.

```
1 if condition_1:
2     block_1
3 elif condition_2:
4     block_2
5 elif condition_3:
6     block_3
7 else:
8     fallback_block
```



if-elif-else Example 1: Grading System

```
1 marks = float(input("Enter marks: "))
2
3 if marks >= 80:
4     print("Grade: A+")
5 elif marks >= 70:
6     print("Grade: A")
7 elif marks >= 60:
8     print("Grade: B")
9 elif marks >= 40:
10    print("Grade: Pass")
11 else:
12    print("Fail")
```



if-elif-else Example 2: Temperature Status

```
1 temp = float(input("Enter machine temperature: "))
2
3 if temp >= 100:
4     print("CRITICAL: Shut down immediately")
5 elif temp >= 80:
6     print("WARNING: Cooling needed")
7 elif temp >= 50:
8     print("Normal operating range")
9 else:
10    print("Low temperature")
```

- **ORDER MATTERS**

Check the most specific or most serious condition first.



if-elif-else Example 3: Simple Calculator

```
1 a = float(input("First number: "))
2 b = float(input("Second number: "))
3 op = input("Operator (+, -, *, /): ")
4
5 if op == "+":
6     print(a + b)
7 elif op == "-":
8     print(a - b)
9 elif op == "*":
10    print(a * b)
11 elif op == "/":
12    print(a / b)
13 else:
14    print("Invalid operator")
```



if-elif-else Example 4: BMI Category

```
1 bmi = float(input("Enter BMI: "))
2
3 if bmi < 18.5:
4     print("Underweight")
5 elif bmi < 25:
6     print("Normal")
7 elif bmi < 30:
8     print("Overweight")
9 else:
10    print("Obese")
```

• NOTICE

We do not need **bmi >= 18.5 and bmi < 25**. Previous failed conditions already handle the lower bound.



Combined Logic Example 1: Admission Rule

```
1 gpa = float(input("GPA: "))
2 english_score = int(input("English score: "))
3
4 if gpa >= 3.5 and english_score >= 70:
5     print("Eligible for admission")
6 else:
7     print("Not eligible")
```

- USING AND

Both requirements must be satisfied.



Combined Logic Example 2: Weekend or Holiday

```
1 day = input("Enter day: ")
2 is_holiday = input("Holiday? yes/no: ")
3
4 if day == "Friday" or is_holiday == "yes":
5     print("No class today")
6 else:
7     print("Class day")
```

- USING OR

At least one condition is enough.



Nested Conditions

What is nesting?

Nesting means writing an **if** statement inside another **if** statement.

```
1 username = input("Username: ")
2 password = input("Password: ")
3
4 if username == "admin":
5     if password == "1234":
6         print("Welcome admin")
7     else:
8         print("Wrong password")
9 else:
10    print("Unknown user")
```



Nested Conditions: Number Sign

```
1 num = int(input("Enter number: "))
2
3 if num >= 0:
4     if num == 0:
5         print("Zero")
6     else:
7         print("Positive")
8 else:
9     print("Negative")
```

- TIP

Too much nesting makes code harder to read. Use **elif** when possible.



— PRACTICE

Common Beginner Mistakes

Watch these carefully

1 Writing `=` instead of `==` in a condition.

2 Forgetting the colon after `if`, `elif`, or `else`.

3 Incorrect indentation.

4 Checking conditions in the wrong order.

5 Using `and` when `or` is needed, or the opposite.



— PRACTICE

Practice 1: Find the Output

```
1 x = 15
2
3 if x > 10:
4     print("A")
5 if x > 20:
6     print("B")
7 else:
8     print("C")
```

• QUESTION

What will be printed? Why?



— PRACTICE

Practice 2: Write the Condition

Write a program that checks whether a number is between 1 and 100.

```
1 num = int(input("Enter number: "))
2
3 # complete the condition
4 if _____:
5     print("Valid range")
6 else:
7     print("Out of range")
```

• HINT

Use **and**.



Practice 3: Fix the Error

```
1 marks = int(input("Enter marks: "))
2
3 if marks => 40
4 print("Pass")
5 else
6 print("Fail")
```

- **FIND AND FIX**

There are comparison, colon, and indentation errors.



— PRACTICE

Practice 4: Build a Mini ATM

Task: Ask for balance and withdrawal amount.

- If withdrawal amount is greater than balance, print **"Insufficient balance"**.
- If amount is less than or equal to zero, print **"Invalid amount"**.
- Otherwise, subtract and print new balance.

```
1 balance = float(input("Balance: "))
2 amount = float(input("Withdraw amount: "))
3
4 # write your branching logic here
```



— WRAP-UP

Summary

Core takeaways

i

Branching helps programs make decisions.

ii

Conditions evaluate to **True** or **False**.

iii

Use **if** for one-way decisions.

iv

Use **if-else** for two-way decisions.

v

Use **if-elif-else** for many possible outcomes.



Recommended Class Flow

Lecture

- Explain concept with real-life examples.
- Run 2–3 examples live.
- Ask students to predict output.

Lab

- Voting eligibility.
- Grading system.
- Calculator.
- Mini ATM task.

— WRAP-UP

Thank *You!*

Questions?

Practice at home: **branching** until it feels natural.